Ders 10 Genişletilmiş Notu: Large Class, Divergent Change, Shotgun Surgery, Primitive Obsession, Feature Envy ve Video Kiralama Örneğinin Adım Adım Refactoring'i

Ders 10 Genişletilmiş Notu: Large Class, Divergent Change, Shotgun Surgery, Primitive Obsession, Feature Envy ve Video Kiralama Örneğinin Adım Adım Refactoring'i

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Large Class: Aşırı Büyük Sınıf
- 2. Extract Class: Sınıf Çıkartma
- 3. Extract Subclass: Alt Sınıf Çıkartma
- 4. Duplicate Observed Data: Gözlenen Verinin Çoğullanması
- 5. Divergent Change ve Shotgun Surgery
- 6. Move Method: Metot Taşınması
- 7. Primitive Obsession: İlkel Tiplerin Aşırı Kullanımı
- 8. Replace Data Value with Object
- 9. Replace Type Code with Class
- 10. Replace Type Code with Subclasses
- 11. Replace Type Code with State/Strategy
- 12. Replace Array with Object
- 13. Feature Envy: Özellik Kıskançlığı
- 14. Video Kiralama Örneği: Başlangıç Durumu
- 15. Birinci ve İkinci Müdahale: Extract Method ve Anlamlı Adlar
- 16. Üçüncü Müdahale: amountFor Metodunu Rental'a Taşımak
- 17. Testlerde Kara Kutu ve Beyaz Kutu Yaklaşımı
- 18. Dördüncü Müdahale: Replace Temp with Query ve Başarım Tartışması
- 19. Beşinci ve Altıncı Müdahale: Bonus ve Toplamlar
- 20. Yedinci ve Sekizinci Müdahale: Hesabı Movie'ye Taşımak
- 21. Dokuzuncu Müdahale: Katmanlı İletişim ve Son Ayıklamalar
- 22. Onuncu Müdahale: Çok Biçimlilik ve Geriye Doğru Uyumluluk
- 23. Karşıt Kalıplar (Antipatterns)
- Örnek Sorular
- Dersin Toparlaması

1. Large Class: Aşırı Büyük Sınıf

Ders, önceki derste kalınan yerden, kod kusurlarının devamıyla başladı. Bu derste işlenen slaytlar NDTM-2024-020 dosyasının 31. slaytından başlar (ekrandan, 00:10:00).

İlk kusur Large Class'tır; hoca bunu "aşırı büyük sınıf" diye çevirdi. Büyüklükten kastedilen öncelikle kod satırı (line of code) cinsinden büyüklüktür. Slayttaki tanım (ekrandan, 00:10:00):

- Sınıfın büyük olması, sınıfa ilgisiz sorumluluklar atandığının göstergesidir. Böylece uyum düşecektir.
- Üye alan sayısının fazlalığı, aşırı büyük sınıfın bir göstergesidir.
- Çözüme götürecek düzenlemeler:
  - Extract Class: birlikte anlam ifade eden üye alanlardan bir veya daha fazla yeni sınıf oluşturulur.
  - Extract Subclass: bazen oluşturulacak yeni sınıfın, mevcut sınıfın bir alt sınıfı olması daha uygun olabilir. Örneğin mevcut sınıf bazı üyelerini her zaman kullanmıyorsa, bunlar yeni bir alt sınıfa aktarılabilir.
  - Duplicate Observed Data: iş mantığı ile kullanıcı arayüzü arasında yüksek bağımlılık varsa GUI sınıfları çok büyüyecektir. Bu durumda veri yeni bir uygulama alanı sınıfına kopyalanır ve aradaki eşgüdüm Observer tasarım kalıbına göre sağlanır.

Hoca sınıfın çok sorumluluk taşımasını iki yönden ele aldı. Sorumluluklar birbiriyle büyük olasılıkla ilgisiz olacaktır; bu uyumun düşük olduğu anlamına gelir. Sorumluluklar birbiriyle ilgili olsa bile çok sayıda sorumluluk taşıyan sınıf hantal, anlaşılması zor ve karmaşıktır (00:07:24). Göstergeler yalnız satır sayısı değildir: nesneye yönelimin istediği gibi küçük küçük metotlar yazılmış olsa bile bu küçük metotlar çok fazlaysa sınıf yine Large Class sayılır. Sınıf çok sayıda üye alan içeriyorsa, bu alanların erişim metotları da sınıfı şişirir (00:07:40).

2. Extract Class: Sınıf Çıkartma

Extract Class, birbiriyle ilgisiz ya da ilgisi uzak iki iş yapan bir sınıfı, bu iş kümelerine göre bölmektir (00:12:24). İkinci işle ilgili kısım seçilip yeni bir sınıfa taşınır. Mevcut sınıfın birbiriyle yakından ilişkili üye alanlarından bir alt küme alınır ve yeni bir sınıf oluşturulur; o üye alanları en yoğun kullanan metotlar da yeni sınıfa çekilir (00:08:34).

Kod kesilip atılmaz, yalnız yeni bir yere taşınır. Mevcut sınıfın bazı işleri için yeni sınıfa yine ihtiyaç duyulacağından, mevcut sınıftan yeni sınıfa genelde bir sahiplik ilişkisi kurulur. Hoca bunu “iletişimi kesmeyecek bir köprü” diye anlattı (00:09:28). Bir kullanma ilişkisi yetiyorsa o da olur, ama genelde sahiplik gerekir. Hoca bu eylemin yalnız Large Class için değil, başka sorunları çözmek için de kullanılabileceğini ekledi.

2.1 Telefon numarası örneği

Önceki halde `Person` sınıfında ofis alan kodu ve ofis numarası üyeleri ile diğer metotların arasında `getTelephoneNumber` metodu vardır (00:12:48). Bakıldığında alan kodu ve numaranın aslında kişinin telefon numarasına ait bilgiler olduğu, telefon numarasının başlı başına bir varlık olarak ele alınabileceği görülür. Kaldı ki şirketlerin de telefonu olabilir; şirket sınıfında da telefonla ilgili metotlar varsa, sistem geneline bakıldığında bu bir Duplicated Code sorunudur. Extract Class bu sorunu da çözer.

Sonraki halde telefon numarasıyla ilgili veriler `TelephoneNumber` sınıfına alınır; alan kodu ve numara üye alanları oraya taşınır. `getTelephoneNumber` metodunun gövdesi de oraya taşınır, ama bir kişinin telefon numarasına erişme ihtiyacı sürdüğü için `Person` sınıfından tamamen silinmez. `Person.getTelephoneNumber` metodunun tek işi, `officeTelephone` adlı yeni üye alanın `getTelephoneNumber` metodunu çağırıp sonucu iletmek olur (00:14:12).

> **Hocanın vurgusu:** Yeni sınıfa taşınan metotların gövdeleri eski sınıftan tamamen silinmez. Eski sınıfta, çağrıyı yeni yere iletmekten sorumlu küçük bir metot bırakılır (00:14:34).

Telefonla ilgili başka işe yarar metotlar da bu sınıfa konabilir. Hoca örnek olarak, birden fazla telefon numarası varsa aynı alan koduna sahip olanları bulmayı ya da hangi operatörün hattını kullanmanın daha avantajlı olacağını hesaplamayı saydı (00:15:15).

3. Extract Subclass: Alt Sınıf Çıkartma

Slayttaki tanım (ekrandan, 00:16:00): bir sınıfın bazı üyeleri her zaman kullanılmıyorsa, bu üyelerden yeni bir alt sınıf oluşturmak daha doğru olacaktır.

Örnek:

```
ÖNCE:
JobItem
  getTotalPrice
  getUnitPrice
  getEmployee

SONRA:
JobItem                  (üst sınıf)
  getTotalPrice
  getUnitPrice
    ^
    | (kalıtım)
LaborItem                (alt sınıf)
  getUnitPrice
  getEmployee
```

Hoca senaryoyu şöyle anlattı (00:15:35): bazı işler dışarıya yaptırılır ve kimin yaptığı önemli değildir; bazı işler ise konusunda uzman bir ustaya, yani belirli bir çalışana (employee) atanır. Sınıfta iki farklı tür iş olduğu fark edilince, belirli bir çalışana yaptırılan işleri temsil etmek üzere `LaborItem` alt sınıfı oluşturulur ve `getEmployee` oraya konur.

Geriye doğru uyumluluk açısından daha hassas olmak gerekiyorsa, yani sistemin bir yerlerinde hâlâ `JobItem.getEmployee` metodunu çağıran kod varsa, metot `JobItem`'dan kaldırılmaz; Extract Class'taki yaklaşım burada da uygulanır (00:16:26). Çalışanı olmayan bir `JobItem` çalışan döndüremeyeceğinden bu metot `null` döndürür; `LaborItem.getEmployee` ise ilgili çalışanı döndürür. Hoca üst sınıfta yalnız `return null` diyen bir metodun çok doğru olmayabileceğini, ilişkideki diğer sınıfları düzenlemek pahalı değilse `getEmployee`'nin `JobItem`'dan tamamen kaldırılmasını önerdi (00:17:10).

4. Duplicate Observed Data: Gözlenen Verinin Çoğullanması

Large Class'ın üçüncü çözümü Duplicate Observed Data'dır; hoca bunu biraz karmaşık bir refactoring eylemi olarak niteledi (00:10:51). Bazen yanlış tasarım yüzünden yazılım katmanlara iyi ayrılamaz; özellikle kullanıcı arayüzü ile iş mantığı katmanları arasındaki bağımlılık çok yüksek olur. Bu çoğunlukla veri giriş ekranlarında görülür: "bu bilgi hazır, hemen kullanalım" yaklaşımı, saf olarak ön yüzden sorumlu olması gereken sınıfların iş mantığıyla da uğraşmasına yol açar. Bu tam bir uyum eksikliğidir; birbirinden ayrı olması gereken iki alandaki sorumluluklar tek sınıftadır (00:11:37).

Slayttaki tanım (ekrandan, 00:20:00):

- Uygulama alanı ile ilgili bir verinin sadece bir GUI sınıfında saklanması doğru değildir.
- Bu veri uygulama alanı/bilgi düzeyi katmanında oluşturulacak yeni bir sınıfa taşınmalı ve,
- GUI nesnesi ile yeni bilgi nesnesi arasında Observer tasarım kalıbı kurulmalıdır.

Slayttaki örnek (ekrandan, 00:20:00):

```
ÖNCE:
IntervalWindow
  startField: TextField
  endField: TextField
  lengthField: TextField
  StartField_FocusLost
  EndField_FocusLost
  LengthField_FocusLost
  calculateLength
  calculateEnd

SONRA:
IntervalWindow  - - - -|>  <<interface>> Observer
  startField: TextField
  endField: TextField
```

```
  lengthField: TextField
  StartField_FocusLost
  EndField_FocusLost
  LengthField_FocusLost
      |
      | 1
      v
Interval  ------|>  Observable
  start: String
  end: String
  length: String
  calculateLength
  calculateEnd
```

Önceki halde alanlara odaklanıldığında ya da odaktan çıkıldığında (focus kazanma ve kaybetme) çalışan metotlar GUI olaylarıyla ilgilidir. `calculateEnd` ise iş mantığıdır; hoca bunun bir abonelik süresi ya da prim yatırılan gün sayısı gibi bir şey hesaplayabileceğini söyledi (00:18:39). Bu pencere sınıfı bir GUI çerçevesinin bir sınıfından türetilmiştir, ama hem GUI işlerini hem iş mantığını yapar.

Sonraki halde `calculateLength` ve `calculateEnd` gibi iş mantığı metotları ile `start`, `end`, `length` üyeleri yeni `Interval` sınıfına alınır. `IntervalWindow` saf GUI sınıfı olur ve aralarına bir sahiplik ilişkisi konur. Pencere sınıflarına ek üye alan eklemek mümkündür. Bir pencere bir ya da birkaç varlığa bilgi topluyorsa o varlıklarla sahiplik ilişkisi içermeli, o verilerin alanlarını doğrudan içermemelidir (00:19:59). `FocusLost` gibi olaylardan sonra yapılacak işler, bu 1 çokluklu sahiplik kapsamındaki `Interval` üyesi üzerinden kodlanır.

> **Hocanın vurgusu:** Sınıf şemasında sahiplik ilişkisi okla gösterildiğinde `IntervalWindow`'un gövdesinde `Interval` türünden bir üye alan ayrıca gösterilmese de vardır. Hoca bunu "redundant information", kendi deyişiyle "gizli bilgi" diye andı: hem okla hem sınıf gövdesinde anlatılabilen bir olgu, yalnız birinde gösterilmişse ötekinde de var demektir (00:20:42). Kod yazarken buna göre davranılır.

4.1 Observer'a ne zaman gerek var?

Hoca yeni veri girildiğinde ya da veri güncellendiğinde ne olacağını tartıştı (00:21:34). Normal veri girişinde Observer'a gerek yoktur: GUI sınıfı olayları zaten önce kendisi yakalar ve iş nesnesini günceller. Observer, iş mantığındaki veri değiştiğinde GUI'nin kendiliğinden güncellenmesi isteniyorsa gerekir. Hoca buna, ayrı süreçlerde ya da ayrı konteynerlerde çalışan dağıtık sistemleri ve raporlama ekranlarını örnek verdi (00:22:46). Bu senaryoda gözleyici (Observer) pencere, gözlemlenen (Observable) ise `Interval` nesnesidir; slayttaki şema da bunu gösterir.

> **Ders sonrası not:** Hoca bu kısımda gözleyici ve gözlemlenen rollerini bir an karıştırdı ve hemen düzeltti (00:22:02-00:22:46). Doğrusu slayttaki gibidir: `IntervalWindow` `Observer` arayüzünü gerçekler, `Interval` `Observable`'dan türer.

5. Divergent Change ve Shotgun Surgery

Hoca bu iki kusuru birlikte anlattı; ortak noktaları değişikliklerin farklı noktalarda yapılmasının gerekmesidir (00:23:58). Slayttaki tanım (ekrandan, 00:24:30):

- Gereksinimlerde bir düzenleme olduğunda veya bir hatayı düzeltmek için kodda değişiklikler yapılması gerekecektir.
- Bu değişikliklerin mümkün olduğunca az yerde yapılması tercih edilir.
- Halbuki bir gereksinim değişikliğini karşılamak için gerekli kod değişiklikleri aynı sınıfın farklı metotlarında yapılıyorsa (divergent change) veya birden fazla sınıfta yapılıyorsa (shotgun surgery), bu işte bir terslik var demektir.
- Divergent change için kötü koku örneği: yeni bir veritabanı ile çalışmak gerekirse şu sınıfın şu 3 metodu değişir.
- Shotgun surgery için kötü koku örneği: yeni bir tür kredi söz konusu olunca şu 2 sınıftaki şu 5 metot değişmeli.

- Çözüme götürecek düzenlemeler:
  - Extract Class: söz konusu metotlar ve ilgili üyelerden yeni bir sınıf çıkartılır.
  - Move Method / Move Field: söz konusu üyeler daha uygun bir mevcut sınıfa taşınır.

Hoca hedefi şöyle açıkladı (00:24:17): değişiklikleri az yerde yapabiliyorsak tasarımı doğru yapmışızdır; yani kendi içinde uyumu yüksek, bağlaşımı düşük sınıflar elde etmişizdir. Böyle bir tasarımda bir değişiklik isteği en fazla bir iki sınıfı etkiler, beş altı sınıfta değişiklik gerekmez; ya da bir sınıfın bir iki metodu değişir, beş on metodu değişmez. Sözlü örnekte veritabanı MySQL'den MSSQL'e (ya da tersine) geçirildiğinde bir sınıfın 3, hatta 5 farklı metodunun değiştirilmesi gerekmesi anıldı (00:25:37).

5.1 Saçma tüfeği benzetmesi

Hoca Shotgun Surgery adını bir benzetmeyle anlattı (00:26:17). Shotgun, çifte tüfektir; normal mermi değil saçma atar. Büyük bir kartuşun içinde çok küçük demir ya da başka maddeden parçacıklar bulunur ve ateş edilince geniş bir alana yayılırlar. Normal mermi ise tek bir noktayı bulur. Bir hedef tahtasının üzerine sınıf şeması resmi konsa, saçmayla ateş edildiğinde çok sayıda sınıfa delik açılır; tabancayla ateş edildiğinde ise en fazla bir sınıfın birkaç metoduna isabet edilir. Hoca bu iki kusur için uygun bir Türkçe ad bulamadığını ("çifteli ameliyat" demenin de anlamsız kaçacağını) söyledi ve İngilizce adlarıyla kullandı.

6. Move Method: Metot Taşınması

Slayttaki tanım (ekrandan, 00:30:00):

- Bir metot tanımlandığı sınıf içinden çok, başka bir sınıflardan çağrılıyorsa; en çok hangi sınıftan çağrılıyorsa o sınıfa taşınmalıdır.
- Bu metot ya eski sınıfından tamamen silinir, ya da eski sınıftaki hali sadece yeni sınıftaki halini çağıracak şekilde değiştirilir (delegation).
- Bu düzenleme de pek çok kokuyu giderdiğinden çok sık kullanılır ve bazı düzenlemelerin alt adımıdır (Ör: Sınıf çıkartma/Extract Class).
- Örnek: önce `Class 1` içinde `aMethod()` vardır, `Class 2` boştur; sonra `aMethod()` `Class 2`'ye taşınmıştır.

Hocanın sayısal örneği (00:28:43): bir metot kendi sınıfının içinden 2 ayrı yerden, X sınıfından 5 ayrı metotta, Y sınıfından 7 ayrı metotta, Z sınıfından 3 ayrı metotta çağrılıyorsa, en çok Y sınıfından çağrıldığı için metodun olması gereken yer Y sınıfıdır. Taşırken, Extract Class'ta olduğu gibi eski yerden de ihtiyaç sürdüğü için çağrıyı yeni yere iletecek düzenleme yapılır.

Hoca bu iletme işinin adının delegation olduğunu hatırlattı (00:29:41): işi başkasına aktarmak, başkasının yapmasını sağlamak; vekil değil, görevlendirme anlamında. Move Method daha büyük eylemlerin bir adımı olarak da düşünülebilir: Extract Class yapılırken bazı üye alanlarla birlikte onlara erişen metotlar da yeni sınıfa taşınır. Extract Class yapılmıyorsa yapılan iş tek başına bir Move Method olur.

7. Primitive Obsession: İlkel Tiplerin Aşırı Kullanımı

"Obsession" takıntı demektir; "primitive" ise ilkel üyelerdir (00:30:57). Sınıflar ilkel olmayan (non-primitive) türlerdir; int, char, boolean gibi herhangi bir türden tek birimlik veri ise literatürde ilkel (primitive) diye anılır. Uzun süre yapısal programlama yapmış kişiler nesneye yönelime geçtiklerinde, ilkellerin olmaması gereken yerlerde yapısal programlama alışkanlıklarıyla onları kullanmaya devam ederler.

Slayttaki tanım (ekrandan, 00:31:30):

- Yapısal programlamadan gelenler, ilkelleri aşırı yoğun olarak kullanır.
- Bu yoğun kullanımın farklı biçimlerini kelimeler ile tarif etmektense, her birine yönelik yeni düzenlemeleri incelemek tercih edilmiştir.
- Çözüme götürecek düzenlemeler:
  - Replace Data Value with Object
  - Replace Type Code with Class
  - Replace Type Code with Subclasses

- Replace Type Code with State/Strategy
- Replace Array with Object
- Extract Class ve Introduce Parameter Object düzenlemeleri de daha nesneye yönelimli bir program hazırlamak için kullanılabilir.

Hoca, bu kusuru sözle anlatmanın verimsiz olacağını, her vakanın farklı bir eylemle çözüldüğünü, bu yüzden eylemleri görüp her birinin hangi takıntıyı çözdüğünü o sırada söyleyeceğini belirtti (00:31:39). Eylemlerin hepsi "Replace" kelimesiyle başlar.

8. Replace Data Value with Object

Slayttaki tanım (ekrandan, 00:34:30): bir üye alan, hele ki bu alan ile ilgili çeşitli davranışlar söz konusu ise, bir sınıf ile simgelenmelidir.

Hoca bunu şöyle açtı (00:32:52): bir üye yalnız bir ilkelle ya da basit bir ilkel olmayan türle modellenmiştir, ama bu üyeyle ilgili yapılması gereken çok iş vardır. Bu işleri dışarıdan yapmak yerine üyenin kendi içinde yapmak daha anlamlıdır. O zaman üye bir ilkelle değil, bir sınıfla simgelenmelidir. `String` aslında ilkel değildir, ama yapısal programlamada bir `char` dizisi olarak saklandığı düşünülürse ilkel gibi değerlendirilebilir (00:33:28).

Slayttaki örnek (ekrandan, 00:34:30):

```
ÖNCE:
Order
  customer: String

SONRA:
Customer                Order
  name: String  <-->    (aralarında ilişki, slaytta "1" çokluğu)
```

- Programın tümü incelendiğinde, belli bir müşterinin tüm siparişlerinin aranması gibi işlemler yapıldığı görülecektir.
- Bu işlemler çoğaldıkça, Müşteri sınıfı ayrılmadığı sürece, Sipariş sınıfı aşırı büyüyecektir.
- İyisi mi şimdiden bu sınıfları ayıralım.

Hoca senaryoyu genişletti (00:33:43): `Order` sınıfında siparişi veren müşterinin yalnız adı `String` olarak tutuluyorsa, müşteriyle ilgili işlemler gerektiğinde veritabanında bu isme sahip kayıtlar tek tek mi dolaşılacaktır? Bir müşteri yalnız sipariş vermez; ona indirim kuponu verilebilir, e-posta adresini güncelleyebilir. Bunlar ayrı ayrı saklı yordamlarda kodlanırsa aynı varlıkla ilgili veri ve eylemler farklı yerlere dağılır. Bir müşteri yalnız bir isim anahtarıyla tanımlanacak kadar basit değildir; `name` üye alanlı bir `Customer` sınıfı oluşturulur. Böylece `Order`'dan siparişi veren müşteriye tek bir işaretçi izlenerek erişilir; müşteri tablosunda isim araması yapmak gerekmez (00:36:19).

Hoca slayttaki ilişkinin çokluk gösterimini derste düzeltti (00:34:58-00:35:53): bir sipariş yalnız bir müşteri tarafından verilir, bir müşterinin ise bir ya da birden fazla siparişi olabilir; bu yüzden "1" çokluğunun yerini değiştirdi.

> **Ek bilgi (derste anlatılmadı):** Fowler'ın Refactoring kitabındaki bu örnekte, büyük olasılıkla, `Order` sınıfı bir `Customer` nesnesine referans tutar; yani ilişki `Order`'dan `Customer`'a yönlüdür ve `Customer` tarafında çokluk 1'dir.

Hoca `String` yerine int türünde bir alan da olabileceğini söyledi (00:36:57): örneğin müşteri vergi numarası `int` olarak tutulursa bu tam bir ilkeldir ve müşteriyi bulmak yine uzun bir iş olur.

9. Replace Type Code with Class

Slayttaki tanım (ekrandan, 00:40:00): yapısal programlamada sabit değişkenlerin kullanılması alışkanlığı ile ilgilidir. Hoca bu alışkanlığın yapısal programlamada önerildiğini, nesneye yönelimde ise alternatiflerinin önerildiğini söyledi (00:37:33). Bu daha çok enum mantığında işleyen şeylerdir.

Slayttaki örnek (ekrandan, 00:40:00):

```
ÖNCE:
Person
  O: int          (static)
  A: int          (static)
  B: int          (static)
  AB: int         (static)
  bloodGroup: int

SONRA:
Person
   |
   | 1
   v
BloodGroup
  O: BloodGroup   (static)
  A: BloodGroup   (static)
  B: BloodGroup   (static)
  AB: BloodGroup  (static)
```

Slaytta ayrıca şu not vardır: “Ya da kısaca enum kullanabilirsiniz. Enum’u bilmeyen yoksa sonraki 2 yansıyı atlayabiliriz.”

Hoca önceki hali şöyle anlattı (00:37:50): kişilerin kan grubunu saklamak için `Person` sınıfına `public final static` dört üye alan (`O`, `A`, `B`, `AB`) konur. Her `Person` nesnesinde kan grubunu tutmak için `int` türünden, `private` ve static olmayan normal bir `bloodGroup` üyesi tutulur. Örneğin O’yu 0, A’yı 1, B’yi 2, AB’yi 3 ile simgelersek, `person.setBloodGroup(0)` denebilir. Asıl sorun şudur: bir `Person` nesnesinin `bloodGroup` üyesine 10 değeri atanabilir. Böyle bir durumu engellemek için setter metodunda ayrıca kontrol yapmak gerekir (00:39:40). Her birim veri için ayrıca sabitler tanımlamak da çok mantıklı değildir.

Sonraki halde kan grubuyla ilgili üye alanlar ayrılır; bu aslında bir tür Extract Class’tır (00:40:24). `BloodGroup` sınıfı oluşturulur ve `O`, `A`, `B`, `AB` oraya aktarılır, ama artık `int` türünden değil, `BloodGroup` türündendir. Böylece pratik olarak bir enum yapısı kodlanmış olur. Hoca, üye alanı ve üye metodu olmayan basit enum yapılarının da arka planda bu mantıkla desteklendiğini söyledi (00:40:40). Hoca enum yapacaksak doğrudan enum kullanmanın daha doğru olacağını, ama enum kullanmak istemiyorsak bari bu hale getirmek gerektiğini vurguladı (00:40:05).

10. Replace Type Code with Subclasses

Slayttaki tanım (ekrandan, 00:44:00): bir sınıfın davranışını belirleyen bir ilkel tip kodu yerine kalıtımın kullanımı daha uygun olacaktır.

Slayttaki örnek (ekrandan, 00:44:00):

```
ÖNCE:
Employee
  ENGINEER: int   (static)
  SALESMAN: int   (static)
  type: int

SONRA:
Employee
   ^
   | (kalıtım)
   +----------+
   |          |
Engineer   Salesman
```

Hoca önceki hali anlattı (00:41:54): çalışanlar ya mühendis ya da satış elemanı olabilir. `public final static int ENGINEER = 1;` ve `public final static int SALESMAN = 2;` tip kodları tanımlanır ve `Employee` nesnesinde bir `type` üyesi tutulur. Kodun her yerine 1, 2 yazılmaz;

`Employee.ENGINEER` ya da `Employee.SALESMAN` denir ki hangi sayının hangi türe denk geldiğini akılda tutmak gerekmesin. Ama çalışana bir iş yaptırılacağı zaman ilgili bütün metotlarda `switch (type) { case ENGINEER: ... case SALESMAN: ... }` ya da `if (type == ENGINEER) ... else if ...` yapısı görülür (00:42:48).

Yeni çalışan türleri çıktıkça işler daha da uzar: yeni case'ler ve yeni else-if'ler eklenir, `Employee` aynı zamanda bir Large Class, metotları Long Method olur. Hoca bunların alt problem olduğunu, kök nedenin tip kodu kullanmak olduğunu söyledi (00:44:08). Bunu düzeltmenin yolu bu örnekte alt sınıf oluşturmaktır. Hoca bir önceki örnekte (kan grubu) böyle bir if-else durumu olmadığını, bu örnekte olduğunu belirtti.

Switch'in bir default case'i ya da if-else zincirinin sonunda bir else dalı varsa, yani "düz çalışan" için bir davranış varsa, `Employee` sınıfı normal bir sınıf olarak kalır. Default ya da son else yoksa `Employee` soyut (abstract) sınıfa çekilebilir (00:44:40). Bu örnekte default case olmadığı için `Employee` soyuttur. Her farklı tür (mühendis, satış elemanı, yönetici) bu sınıfın alt sınıfları olur. Burada delegation metodu eklemeye gerek yoktur: kalıtımda alt sınıftan bir nesne her zaman üst sınıftan bir nesne yerine kullanılabilir (00:45:19). Yeni çalışan türleri gerektikçe yalnız yeni alt sınıf eklenir.

11. Replace Type Code with State/Strategy

Slayttaki tanım (ekrandan, 00:50:00): önceki örnekte kalıtım kullanımı bir nedenle mümkün değilse, State veya Strategy tasarım kalıbı da kullanılabilir.

Slayttaki örnek (ekrandan, 00:50:00):

```
ÖNCE:
Employee
  ENGINEER: int   (static)
  SALESMAN: int   (static)
  type: int

SONRA:
Employee  ---- 1 ---->  EmployeeType
                              ^
                              | (kalıtım)
                         +----------+
                         |          |
                    Engineer   Salesman
```

Hoca kalıtımın kullanılamayacağı durumları saydı (00:46:54). Çoklu kalıtım her dilde desteklenmez; `Employee` zaten başka bir sınıfın alt sınıfıysa ek bir kalıtım hiyerarşisi kurulamayabilir. Bir başka neden: bazı işler `Engineer` için anlamlıyken `Salesman` için hiç anlamlı olmayabilir (00:49:12). Hoca bunu Composite kalıbıyla hatırlattı: yaprak düğümlere çocuk eklemenin anlamı yoktu ve bu metodun gövdesi yalnız açılış ve kapanış parantezinden oluşan boş bir gövdeyle kodlanıyordu. Kalıtımla bölmek benzer durumlara yol açıyorsa State ya da Strategy kurulur; o zaman o işleri anlamsız yerlerde kodlamak gerekmez. Bundan kurtulunamasa bile, iş mantığını oluşturan sınıftan kalıtım yerine yalnız işin yapılış biçimini özelleştirmek daha mantıklı gelebilir (00:50:03).

Hoca iki kalıbı hatırlattı (00:47:55): Strategy'de bir işi yaptıracak bir nesne verilir, farklı algoritmalar alt sınıflar olarak kodlanır. State'te ise bir durumdan başka bir duruma geçmek gereken vakalar kodlanabilir. Durum geçişi varsa tasarım State, yoksa Strategy şeklinde yapılır.

12. Replace Array with Object

Primitive Obsession'ın son hali Replace Array with Object'tir (00:50:44). Slayttaki tanım (ekrandan, 00:52:30): çok boyutlu dizilerde kayıt tutmak yerine, tek bir nesne kullanın.

Slayttaki örnek (ekrandan, 00:52:30):

```
/*** ÖNCE: ***/
String[][] scores = new
    String[3][teamCount];
scores[0][0] = "Liverpool";
scores[1][0] = "15";
scores[2][0] = "67";

/*** SONRA: ***/
Score[teamCount] scores;
scores[0] = new
    Score("Liverpool");
scores[0].setPoints("15");
scores[0].setAverage ("67");
```

Kod slayttan birebir aktarılmıştır, derlenmedi.

> **Ders sonrası not:** Slayttaki "SONRA" kodunda `Score[teamCount] scores;` Java'da geçerli bir dizi tanımı değildir; büyük olasılıkla `Score[] scores = new Score[teamCount];` kastedilmiştir. Ayrıca hoca `Score` sınıfında puan ve averajın `int` tutulacağını söylediği için (00:53:20) setter'lara büyük olasılıkla `"15"` yerine `15` gibi sayısal değer verilmelidir.

Hoca bu yapının yapısal programlamada, struct yapısı görülmeden önce teşvik edildiğini söyledi (00:50:53). Çok boyutlu dizilerde ya da birden fazla dizide tek bir varlıkla ilgili kayıt tutulabilir. Örnekte her takımın adı, puanı ve averajı tutulur. Hoca anlatırken diziyi her satırı bir takım, her sütunu bir bilgi (ad, puan, averaj) olacak biçimde tarif etti (00:52:23).

Bu yaklaşımın dezavantajları:

- Sayısal veri `String` olarak tutulur; aritmetik işlem yapılacağı zaman önce `int`'e çevirmek gerekir (00:52:36).
- Tek bir çok boyutlu dizi yerine bir boyutlu bir `String` dizisi (takım adları) ve iki bir boyutlu `int` dizisi (puanlar, averajlar) tutulursa, bir varlığın tüm bilgilerine ulaşmak daha da karmaşıklaşır; her şey ayrı ayrı yerlere dağılmış olur (00:52:52).

Bunun yerine `Score` adlı bir sınıf oluşturulur: `String` takım adı, `int` puan ve `int` averaj üye alanlarıyla. Ardından tek bir boyutlu `Score` dizisi tutulur (00:53:20).

Hoca bir ayrım yaptı (00:53:50): soldaki örnekte bir struct tanımlanıp tek boyutlu bir struct dizisi tutulsaydı bu Primitive Obsession olmazdı. Yapısal programlamada ilkel-ilkel olmayan ayrımı olmasa da kod nesneye yönelime taşınırken struct'ın yerine zaten bir sınıf oluşturulacaktır; nesneye yönelimde struct yoktur.

13. Feature Envy: Özellik Kıskançlığı

Ders öncesi kısmın son kod kusuru Feature Envy'dir (00:55:18). "Envy" kıskançlık demektir; burada başkasının üyesini kıskanmak anlamında kullanılır. Hoca "feature" kelimesinin üye alan anlamına geldiğini, ama metotların da kıskanılabileceğini söyledi; bu yüzden çerçeveyi hem üye alanların hem üye metotların kıskanılması diye geniş tuttu (00:56:48). Yalnız bir üye alan kıskanılsa bile ona erişebilmek için o alana erişen metotlar da taşınır; bu sırada Move Method ya da Extract Method yapılır. Hoca, Move Method'u anlatırken neden taşıdığımızı açıkladığı tarifin aslında Feature Envy olduğunu da belirtti (00:56:03).

Slayttaki tanım (ekrandan, 01:00:00):

- Bir metot ait olduğu sınıfın üyelerinden çok başka sınıfların üyelerine erişiyorsa, bu duruma 'Özellik kıskançlığı' adı verilir.
  - 'Çok' sıfatı ile o koddaki ortalama (veya %20 gibi bir eşikten fazla) farklı sınıf üyesi erişim sayısının üzerinde bir sayı ifade edilmektedir.
  - Sorumlulukların doğru atanmadığının göstergesi olabilir.
  - Elbette bağlaşım ilkesi nedeniyle başka sınıflardan nesnelere erişmek gerekecektir, bu nedenle hiçbir sınıf sadece kendi metotlarını kullanmaz. Öyle olsaydı bu kez de uyum ilkesini olumsuz yönde etkilemiş olurduk.
  - Strategy ve Visitor kalıpları bir özellik kıskançlığı durumu olarak yorumlanmamalıdır.

- Çözüm için olası refactoring eylemleri:
  - Move method
  - Extract method: eğer kıskançlık metodun sadece bir kısmında ise o kısmı çıkartıp kıskanılan üyelerin bulunduğu sınıfa taşınabilir.

Hoca “çok” kelimesinin bir rakama dönüştürülmesi gerektiğini söyledi (00:57:42): ortalamanın üzeri ya da kendi belirlenecek beşte bir (%20) gibi bir eşik. Bir metot yapacağı işlerde kendi üyelerinden çok başka sınıfların üyelerini çekiyorsa, aradaki bağlaşım da gereğinden fazla kurulmuş olabilir. İlgili metotlar taşınıp iki sınıf arasındaki ilişkiye artık gerek kalmıyorsa, Feature Envy çözülürken bağlaşım da düşürülmüş olur (00:58:47). Kıskançlık metodun yalnız bir kısmındaysa, metodu komple taşımak yerine o kısmı Extract etmek daha doğrudur (01:00:28).

> **Hocanın vurgusu:** Strategy ve Visitor kalıpları Feature Envy olarak yorumlanmamalıdır (00:59:08). Visitor, parça-bütün ilişkisindeki hiyerarşik bir yapının bütün elemanlarını ziyaret eder ve onlar üzerinde işlem yapar; koda bakıldığında kendi üyelerinden çok başka nesnelerin işlerini yaptırır, ama bu bir kıskançlık değil, bilinçli bir tasarımdır. Strategy’de de iş, verilen alternatif algoritma nesnelerinden birine yaptırılır. Bunlar bir problemi doğru çözmek için yapılan tasarım kalıplarıdır.

Aynı slaytın altında bir sonraki kısma geçiş de yazılıdır (ekrandan, 01:00:00): “Bir tasarımın ardışıl düzenlemeler ile değiştirilmesi: Şimdiye kadar incelenen düzenlemelerden bazılarının birlikte kullanılarak, yapısal programlama yaklaşımı izleri taşıyan bir tasarımın nasıl değiştirildiğinin bir örneği için Fowler’ın Refactoring kitabının 1. bölümü incelenebilir.”

14. Video Kiralama Örneği: Başlangıç Durumu

Aradan sonra hoca, çok kusurlu bir programı küçük refactoring eylemleriyle iyileştiren, Martin Fowler’ın kitabındaki bütüncül örneğe geçti (01:00:45). Uygulama Eclipse’te yapıldı; JUnit testleri hazırdı.

Slayttaki ilk durum (ekrandan, 01:20:00): örneğin kaynak kodu `refactoring/fowler00` paketindedir.

```
Sınıf şeması:
Movie
  - _priceCode: int
     ^ 1
     |
     | 1
Rental
  - _daysRented: int
     ^ *
     |
     | 1
Customer
  + statement()

Sıralama şeması (özet):
Kullanıcı -> aCustomer: statement()
aCustomer: *[for all rentals]
  aCustomer -> aRental: getMovie()
  aCustomer -> aMovie: getPriceCode()
  aCustomer -> aRental: getDaysRented()
    (tek kira için ücret hesabı tamamlanır)
  aCustomer -> aMovie: getPriceCode()
  aCustomer -> aRental: getDaysRented()
    (tek kira için bonus hesabı tamamlanır)
  aCustomer -> aMovie: getTitle()
    (tek kira için makbuz satırı basımı tamamlanır)
(biriken bilgiyle makbuz basımı tamamlanır)
```

Hoca örneği hatırlattı (01:19:36): müşteriler film kiralar. Filmlerin üç farklı fiyat kodu vardır: Regular, New Release ve Childrens; bunlar farklı sürelerde, farklı bedellerle kiralanır. Kiralamalardan bonus puan da kazanılır. `Customer`'ın büyük bir `statement` metodu vardır: hangi filmin kaç günlüğüne, hangi bedelle kiralandığını listeler, sonra toplam borcu ve kazanılan toplam puanı yazan bir makbuz `String`'i oluşturup döndürür. Bunun için birim sınamaları önceden hazırlanmıştı. İş mantığı sınıfları ve test sınıfları bu kez aynı pakette tutuldu.

Hoca her adımı ayrı bir paket kopyasında yaptı: paketi kopyalayıp `fowler_01`, `fowler_02` gibi adlarla yapıştırdı ve değişikliği yeni pakette uyguladı (01:27:08). Böylece her ara sürüm korunur ve sürümler karşılaştırılabilir.

Kod incelendiğinde (01:23:06):

- En uzun sınıf yaklaşık 57 satırlık `Customer`'dır. En büyük, en kusurlu ve en "kokulu" sınıfın bu olduğu tahmin edilebilir.
- `Rental` yalnız 16 satırlık, çok basit bir sınıftır.
- `Movie`'de bir Primitive Obsession vardır: fiyat tip kodu. Replace Type Code ile değiştirilmesi gerekir, ama geriye doğru uyumluluğu korumak için buna en son el atılacaktır.
- `Customer` içinde en uzun metot `statement`'tır; içinde en göze çarpan kısım `switch` bloğudur.

  Hocanın vurgusu: Refactoring ezberden ve doğaçlama yapılmaz. Hoca, eğitimsel ve anlatım sırası açısından değerli olduğu için Fowler'ın izlediği yolu adım adım izledi (01:24:24). Polimorfizm gerektiği açık olsa bile, her şeyi bir anda yeni sınıflara taşımak yerine küçük adımlarla refactoring tercih edilir (01:25:16).

15. Birinci ve İkinci Müdahale: Extract Method ve Anlamlı Adlar

15.1 Birinci müdahale: switch bloğunu çıkarmak

İlk iş, Long Method olan `statement`'ın içindeki `switch` bloğunu Extract Method ile `amountFor` adlı yeni bir metoda çıkarmaktır (01:25:16). Yeni metot belli bir kiralama işleminin bedelini (amount) hesaplar. Hoca bazı refactoring eylemlerini Eclipse'in kendisinin yapabildiğini, ama ders notlarını ilk hazırladığı sırada Eclipse'in bu Extract Method'u düzgün yapamadığını, derleme hataları verdiğini anlattı (01:25:44). Ayrıca ders kapsamında nasıl yapıldığını göstermek için işlemi elle yapmak daha doğrudur.

Başlamadan önce bütün testlerin geçtiği doğrulandı (01:26:40). Metot `private` yapıldı: bir kira kaydının parametre olarak verilmesi gerektiğinden dışarıdan çağrılacak bir metot değildir (01:28:04). Bloğun başındaki "her bir film kirası için bedeli hesapla" yorumu, burada ayrı bir iş yapıldığını gösterir; yeni metot bu işi adıyla anlattığı için yoruma gerek kalmaz (01:28:43).

Kesilip yapıştırılan blokta bilinmeyen değişkenler vardır (01:29:26). Döngüde sıradaki kiralama `each` değişkeninde tutulduğu için `each` yeni metoda parametre olarak verilir. `thisAmount` geçici değişkeni yeni metotta tanımlanır ve sonunda `return thisAmount;` ile döndürülür.

Ekrandaki ara hal (ekrandan, 01:30:00 ve 01:40:00):

```
// statement() içinde:
while (rentals.hasMoreElements()) {
    double thisAmount = 0;
    Rental each = rentals.nextElement();

    thisAmount = amountFor(each);
    // add frequent renter points
    frequentRenterPoints ++;
    // add bonus for a two day new release rental
    if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE)
            && each.getDaysRented() > 1) frequentRenterPoints ++;
    //show figures for this rental
    result += "\t" + each.getMovie().getTitle()+ "\t" + ...
}

// çıkarılan metot:
```

```
private double amountFor( ) {
    switch (each.getMovie().getPriceCode()) {
    case Movie.REGULAR:
        thisAmount += 2.0;
        if (each.getDaysRented() > 2)
            thisAmount += (each.getDaysRented() - 2) * 1.5;
        break;
    case Movie.NEW_RELEASE:
        thisAmount += each.getDaysRented() * 3.0;
        break;
    case Movie.CHILDRENS:
        thisAmount += 1.5;
        if (each.getDaysRented() > 3)
            thisAmount += (each.getDaysRented() - 3) * 1.5;
        ...
```

Kod parçadır, derlenmedi. 01:30:00 karesi, parametre eklenmeden önceki ara hali gösterir (`amountFor( )` boş parametreli ve `each` henüz tanımsız); son halde imza `amountFor(Rental each)` olur.

Testler çalıştırılınca bonusla ilgili bir hata çıktı (01:32:09). Hoca toplam bedeli biriktiren `totalAmount += thisAmount;` satırını kestiğini fark etti ve geri ekledi; ama bu kez satırı iki kez yazdığı için bedeller iki katına çıktı (01:34:01). Fazla satır silinince testler geçti.

> **Hocanın vurgusu:** Hata yapmak kolaydır, ama test case'ler hazırsa ve her adımdan sonra sürekli kontrol ediliyorsa hata hemen yakalanır, bir yerlerde kaybolunmaz (01:34:22).

Ardından hoca Eclipse'in "Compare With" özelliğiyle kendi sürümünü ders notlarındaki sürümle karşılaştırdı (01:35:03). Yalnız boş satırlar, gereksiz bir tür dönüşümü ve sabitlerin ondalık yazılıp yazılmaması gibi küçük farklar vardı. Hoca, kodu sıfırdan yazan kişi daha iyi hatırlayabilse de, satırın neden fazladan eklendiğini hatırlayamamasının bakım zorluğuna bir örnek olduğunu söyledi (01:36:40).

Her müdahaleden sonra sıralama şeması (sequence diagram) yeniden çizilir. Hoca, Fowler'ın daha serbest çizdiği şemaları notlarda daha ayrıntılı, özellikle dönüş oklarına dikkat ederek çizdiğini söyledi (01:37:12).

15.2 İkinci müdahale: değişken adları

`amountFor` içindeki adlar anlamlı değildir (01:37:40). `each` "her bir ne?" sorusunu açık bırakır; bu bir kiralama işlemidir, bu yüzden adı `aRental` olur. Yeni metot yalnız belirli bir kiranın bedelini hesapladığı için `thisAmount` adı da anlamsızdır; adı `result` yapılır. Bu değişiklikler Eclipse'in Refactor > Rename özelliğiyle yapıldı (01:39:01).

Soru: Değişken adını değiştirmek refactoring sayılır mı? **Cevap:** Sayılır; en azından bu örnek için sayılır (01:38:27).

Hoca, `for-each` döngüsünde kiralama nesnesini `aRental` diye adlandırmanın kendi alışkanlığı olduğunu ve yaygın bir uygulama olduğunu söyledi (01:40:51). `statement` içindeki `each` adını ise döngüdeki sıradaki eleman anlamında bıraktı.

16. Üçüncü Müdahale: amountFor Metodunu Rental'a Taşımak

`amountFor` metodu bir `Rental` parametresi alır, onun filmini alır ve filmin tip kodu üzerinden bir şeyler yapar (01:41:44). Hoca bunun iki sorununu gördü: `Customer` sınıfı bir başka sınıfın tip koduna göre iş yapmaktadır ve bu tip kodu Primitive Obsession olarak zaten göze batmaktadır. Ayrıca metot kiranın kaç gün sürdüğüne de erişir. Metot aslında `Customer`'ın hiçbir üyesine erişmez; bu yüzden `Customer`'da durmamalıdır (01:42:57). Bu bir Feature Envy durumudur.

Doğru yer kısmen `Rental`, kısmen `Movie`'dir. Ama adım adım ilerlemek için önce yalnız `Rental`'a taşınır; daha sonra bir kısmı `Movie`'ye alınacaktır (01:43:18). Doğrudan `Movie`'ye taşıyıp sonra bir parçasının `Rental`'da kalması gerektiğini fark etmek, bir seferde iki üç adım atmak olurdu.

`Rental` kendisiyle ilgili sorgulamaları kabul edebilmelidir. Burada durum değişikliği yoktur, yalnız okuma amaçlı erişimler vardır (01:43:46). Metot `Rental`'a taşınırken `private` olmaktan çıkıp `public` yapılır. Parametre artık gerekmez; metot `this` nesnesi üzerinde çalışır, yani `aRental.` önekleri silinir ya da `this.` yapılır (01:44:57). `Customer`'da ise `aRental.amountFor()` çağrılır. Testler sorunsuz geçti.

Ardından metodun adı Eclipse'in Rename özelliğiyle `getCharge` yapıldı: bu kira işleminin bedeli nedir (01:46:25). Hoca yalnız ad değiştirildiği için ayrıca test çalıştırmaya gerek görmedi; hata olsaydı derleme hatası olurdu (01:47:06).

17. Testlerde Kara Kutu ve Beyaz Kutu Yaklaşımı

Bu noktada hoca bir tartışma açtı (01:47:36). Slayttaki metin (ekrandan, 01:50:00):

- Kara kutu sınaması (Black-box testing): sınanacak birimin iç işleyişi bilinmez, sadece birimin beklenen girdilere karşı beklenen çıktıları üretip üretmediğine bakılır.
- Beyaz kutu sınaması (White-box testing): sınanacak birimin iç işleyişi bilinir ve yapılacak sınamalar buna göre belirlenir.
- Tartışma:
  - Üçüncü adımın sonucunda, Rental sınıfı yeni bir metot kazandı.
  - Acaba `Rental.getCharge` metodu için yeni bir test case oluşturalım mı (beyaz kutu), yoksa mevcut test case'ler ile devam mı edelim (kara kutu)?
  - Beyaz kutuyu seçmek test kalitemizi arttırır ancak sınırlı proje zamanımızın uzamasına neden olabilir.
  - Bu aşamada mevcut test case'ler ile devam etmeyi seçtim.

Hoca kara kutuyu, kutunun içini açmadan yalnız doğru girdiye karşı doğru çıktının üretilip üretilmediğine bakmak; beyaz kutuyu ise kutunun içini açıp kodun nasıl çalıştığını incelemek diye anlattı (01:47:48). Yeni bir metot geldiğinde, iç çalışma değiştiği için testleri de değiştirmek beyaz kutu yaklaşımıdır.

Test vakalarının üzerinden geçtiği kod yerleri işaretlenirse bazı yerlerin hiç çalıştırılmadığı görülür (01:49:17). Hoca %100 kod kapsamanın (code coverage) mümkün olmadığını, testin pahalı bir çalışma olduğunu söyledi. Projenin özelliklerine ve kısıtlarına göre %50 ya da %70 kod kapsama gibi bir eşik belirlenir; bu, proje geliştirme etkinliği içinde sınamaya ne kadar adam-zaman ayrılacağına göre seçilir ve aşılmamaya çalışılır (01:49:41). Örnek karmaşık olmadığı için mevcut test case'lerle, yani kara kutuyla devam edildi. Daha karmaşık bir örnekte de, sınama için ayrılan zamanı aşma olasılığı varsa kara kutuyla devam etmek tercih edilebilir (01:50:21).

18. Dördüncü Müdahale: Replace Temp with Query ve Başarım Tartışması

18.1 thisAmount geçici değişkenini kaldırmak

`statement` metodunda `thisAmount` tanımlanır, ona bir kez değer atanır ve değeri bir daha hiç değişmez; yalnız birkaç yerde kullanılır (01:51:16). Değeri hiçbir yerde değişmeyen, yalnız okunan bir geçici değişken silinebilir: geçtiği her yere, ona atanan ifade (`aRental.getCharge()`) yazılır. Böylece Replace Temp with Query eylemi yapılmış olur (01:52:02). Testler geçti.

18.2 Başarım ölçümü

Ardından başarım tartışıldı (01:52:31). Hoca testleri yeni sürümde birkaç kez çalıştırdı; süreler 37, 14, 7 ms gibi değerler gösterdi ve tekrarladıkça 4 ms kadar düştü. Arada görülen sıçramaları (spike) arka planda çalışan işletim sistemi işlerine, antivirüse, güncelleme kontrolüne, açık PowerPoint ve Excel'e bağladı. Önceki sürümde 15, 6, 11, 9 ms gibi değerler gördü (01:53:46). Kesin sonuç için her iki sürümü de en az 10 kez, arka planda mümkün olduğunca az program açıkken çalıştırıp ortalamasını almak gerekir (01:54:07).

Hoca ortalamanın yaklaşık 6 ms'den 10 ms'ye çıktığını söyledi ve aradaki farkın yalnız 4 ms olduğunu, yüzde olarak büyük görünse de hissedilmeyeceğini belirtti (01:54:32).

> **Ders sonrası not:** Hoca 6 ms'den 10 ms'ye çıkışı "yaklaşık üçte bir, %33 başarım azalması" diye nitelendirdi (01:54:32). Süre artışı olarak bakılırsa $(10-6)/6 \approx \%67$; yeni süreye göre bakılırsa $(10-6)/10 = \%40$ eder. Her iki hesapta da oran %33 değildir.

Hoca neden yavaşladığını da gösterdi (01:55:40): `getCharge` döngü içinde her kiralama için çağrılır ve geçici değişken kaldırıldıktan sonra her kiralama için iki kez çağrılır. Hoca 4 kiralama için önce 4 kez çalışan metodun şimdi $4 \times 2 = 8$ kez çalıştığını söyledi (01:56:10).

> **Hocanın vurgusu:** Sırf birkaç milisaniye için bu refactoring eylemini yapmamak doğru değildir; bakım masrafları düşsün diye yapılmalıdır (01:54:51). Ama eylem, özellikle farklı alt sistemler arası çağrılar yüzünden çalışmayı çok uzatıyorsa geri alınabilir (01:54:58).

19. Beşinci ve Altıncı Müdahale: Bonus ve Toplamlar

19.1 Beşinci müdahale: bonus hesabını Rental'a taşımak

Bonus hesabı da kira bedeli gibi `Customer`'dan çok `Rental` ve `Movie` ile ilgilidir; bu yüzden `Rental`'a taşınır (01:56:57). Hoca "frequent renter points" adını Türkçede yerleşmiş "bonus" kelimesiyle karşıladı: bankaların ürünü olarak değil, eski zamanlardan beri bilgisayar oyunlarında kazanılan ek puanlar anlamında (01:57:28).

Mantık da sadeleştirildi (01:58:24): "önce puanı bir artır, şart sağlanırsa bir daha artır" yerine "şart sağlanırsa 2 döndür, yoksa 1 döndür" biçimine çevrildi. Ekrandaki son hal (ekrandan, 02:00:00):

```
public int getBonus( ) {
    if ((getMovie().getPriceCode() == Movie.NEW_RELEASE)
            && getDaysRented() > 1) return 2;
    return 1;
}
```

Kod parçadır, derlenmedi. `Customer`'daki yerel değişkenin adı da Rename ile `bonus` yapıldı ve `bonus += aRental.getBonus();` biçiminde hesaplandı (02:01:17). Hoca bu sırada paketi kopyalamayı unuttuğunu fark edip değişiklikleri geri aldı, `fowler_05` paketini oluşturup işlemi orada yaptı (02:00:08). Testler geçti.

19.2 Altıncı müdahale: toplamları ayrı metotlara çıkarmak

Sırada `totalAmount` ve `frequentRenterPoints` geçici değişkenleri vardır (02:03:57). Uygulamanın bütününde hem toplam bedel hem toplam puan gereklidir, ama bu toplamaların `statement` içinde yapılması gerekmez. Hoca bunları Move Method değil, Extract Method ile `Customer` içinde ayrı metotlara çıkardı (02:05:38). Metotlar `public` yapıldı: kullanıcının toplam bedelinin bilinmesinde sakınca yoktur, bu bilgi zaten faturada geçer (02:06:08). Hoca kümülatif toplam için yine bir geçici değişken gerektiğini, ama bu işin uzun metottan çıkıp ayrı bir alt iş olarak ele alındığını vurguladı (02:07:05).

Ekrandaki hal (ekrandan, 02:09:30):

```
    while (rentals.hasMoreElements()) {
        Rental aRental = rentals.nextElement();
        //show figures for this rental
        result += "\t" + aRental.getMovie().getTitle()+ "\t" +
            String.valueOf(aRental.getCharge()) + "\n";
    }
    //add footer lines
    result += "Amount owed is " + getTotalCharge()+"\n";
    result += "You earned " + getTotalBonus()
        + " frequent renter points";
    return result;
}
public double getTotalCharge( ) {
```

```
    double totalAmount = 0.0;
    for( Rental aRental : _rentals )
        totalAmount += aRental.getCharge();
    return totalAmount;
}
```

Kod parçadır, derlenmedi. Yeni metotlarda `while` + `Enumeration` yerine for-each döngüsü kullanıldı (02:06:38).

Testler çalıştırılınca `testStatementWhenEmpty` kırıldı (02:09:54). Hoca `getTotalBonus`'u `getTotalCharge`'dan kopyala-yapıştırla oluşturduğu için dönüş türünü `double` bırakmıştı; boş makbuzda beklenen "You earned 0 frequent renter points" iken "0.0" üretiliyordu (ekrandan, 02:10:00). Tür `int` yapılıp başlangıç değeri `0` olunca testler geçti. Hoca bunu kopyala-yapıştırın sorunlarına bir örnek olarak gösterdi (02:10:17). Son halde (ekrandan, 02:38:30):

```
public int getTotalBonus( ) {
    int result = 0;
    for( Rental each : _rentals )
        result += each.getBonus();
    return result;
}
```

Hoca güncellenen sınıf şemasında yeni `public` metotların gösterilmesi gerektiğini söyledi (02:13:44). Toplam metotlarının adlandırmasında kendi sürümü ile notlar arasında küçük farklar olduğunu, bunun önemli olmadığını ekledi.

20. Yedinci ve Sekizinci Müdahale: Hesabı Movie'ye Taşımak

20.1 Yedinci müdahale: getCharge'ı bölmek

`getCharge` `Customer`'ın hiçbir üyesine erişmediği için `Rental`'a alınmıştı. Ama `Rental`'da da kendi metotlarının yanında `Movie`'nin tip koduna erişir (02:15:11). Hoca burada hem bir terslik hem de bir `switch-case` sorunu gördü. `switch`'in `Movie`'ye ait bir alan üzerinden kurulması "daha büyük bir ayıp"tır; önce bu çözülür.

Kira bedelini belirleyen iki bilgi vardır (02:16:25): kaç gün kiralandığı (`Rental`'ın verisi) ve filmin türü (`Movie`'nin verisi). Hangi hesap hangi veriyle ilgiliyse o sınıfta durmalıdır. Esas hesap `Movie`'ye yaptırılır: `Rental`, `Movie`'ye "sen şu kadar gün kiralandın" bilgisini verir ve kira bedelini sorar. `Movie` hangi türde olduğuna bakar, parametre olarak gelen gün sayısına göre hesaplayıp bedeli döndürür (02:17:11).

Böylece `Movie`'ye `getCharge(int daysRented)` metodu eklenir. Taşınan kodda `getMovie()` çağrılarına gerek kalmaz, çünkü metot artık filmin kendisindedir; gün sayısı ise `getDaysRented()` yerine parametreden gelir (02:18:51). `Rental.getCharge()` yalnız `_movie.getCharge(_daysRented)` çağrısını döndürür. Hoca gün sayısının üye alan üzerinden ya da `getDaysRented()` metodu üzerinden verilebileceğini, ikisinin de olacağını söyledi (02:20:09). Ekranda Eclipse'in kod tamamlama listesinde önceki paketlerdeki `Movie.REGULAR` sabitleri görünür (ekrandan, 02:20:00). Testler geçti.

Hoca `switch-case`'in hâlâ durduğunu, ama en azından daha makul bir yerde, `Movie` içinde olduğunu söyledi (02:21:55).

20.2 Sekizinci müdahale: getBonus'u bölmek

Aynı mantık bonusa da uygulanır (02:22:58). `Movie`'ye `int` döndüren `getBonus(int daysRented)` eklenir; hoca önceki hataya bu kez düşmemek için dönüş türünün `int` olmasına dikkat çekti. `Rental`'daki kod kesilip `Movie`'ye yapıştırılır; `getMovie()` yerine `this` kullanılır, gün sayısı parametreden gelir. `Rental.getBonus()` artık `_movie.getBonus(_daysRented)` döndürür (02:25:49). Testler geçti.

Slayttaki 8. müdahale sonrası şema (ekrandan, 02:30:00):

```
Movie
  - _priceCode: int
```

```
  + getCharge(daysRented: int): double
  + getBonus(daysRented: int): int
     ^ 1
     | 1
Rental
  - _daysRented: int
  + getCharge(): double
  + getBonus(): int
     ^ *
     | 1
Customer
  + statement()
  + getTotalCharge(): double
  + getTotalBonus(): int

Sıralama şeması (özet):
Kullanıcı -> aCustomer: statement()
  *[for all rentals]
    aCustomer -> aRental: getMovie()
    aCustomer -> aMovie: getTitle()
    aCustomer -> aRental: getCharge()
      aRental -> aMovie: getCharge(_daysRented)
    (tek kira için ücret hesabı ve makbuz satırı basımı tamamlanır)
  aCustomer -> aCustomer: getTotalCharge()
    *[for all rentals] aRental.getCharge()
      aRental -> aMovie: getCharge(_daysRented)
  aCustomer -> aCustomer: getTotalBonus()
    *[for all rentals] aRental.getBonus()
      aRental -> aMovie: getBonus(_daysRented)
  (makbuz basımı tamamlanır)
```

21. Dokuzuncu Müdahale: Katmanlı İletişim ve Son Ayıklamalar

21.1 getMovieTitle

8. müdahaleden sonra sıralama şemasına bakınca hoca bir şey fark etti (02:28:30): `Customer` genelde `Rental` ile, `Rental` da genelde `Movie` ile konuşur. Ama `Customer`'dan doğrudan `Movie`'ye giden tek bir ok kalmıştır: `getTitle`. Makbuzda "şu filmi şu kadar gün kiraladınız" satırı basılırken `Customer`, `Rental`'dan filmi alıp filme adını sormaktadır.

Bu bağlantı kaldırılırsa nesnelerin iletişimi katmanlı olur: `Customer` yalnız `Rental` ile, `Rental` yalnız `Movie` ile konuşur (02:29:25). Hoca bunun tam bir bağlaşım azaltması olmadığını vurguladı (02:29:50): `Customer`'dan `Movie`'ye dolaylı sahiplik ilişkisi her halükarda sürer. Ama nesnelerin konuşmasını katmanlaştırmak daha iyi bir iştir. `Customer`, filmin adını `Rental`'a sorar: "seni kiralamışız, seninle ilişkili filmin adı ne?" Bunun için `Rental`'a `public String getMovieTitle()` eklenir; gövdesi `return _movie.getTitle();` olur (02:30:51).

21.2 addFooter ve statementLine

`statement` hâlâ biraz uzundur ve içinde "add footer lines" ve "show figures for this rental" yorumları vardır (02:32:55). Hoca, kodu yorum bile gerektirmeyecek biçimde yapılandırmanın hedef olduğunu hatırlattı: bir yorum görülüyorsa, bu ayrı bir alt iş olduğunun işaretidir ve kendi metoduna çıkarılabilir (02:33:34). Bu iki iş `fowler_09b` paketinde yapıldı.

Önce `addFooter` çıkarıldı. Döngüdeki hiçbir şeyi kullanmadığı için komple kesilip tek bir `return` ifadesine çevrilebilir; geçici değişkene gerek kalmaz (02:34:53). Sonra makbuz satırı `statementLine` adlı metoda çıkarıldı. `while` döngüsü içinde kiralama nesnesine ihtiyaç olduğu için `aRental` parametre olarak verilir; parametrenin döngüdeki değişkenle aynı adı taşıması kesip yapıştırmayı kolaylaştırdı (02:36:31). Bu iki metot dışarıdan çağrılması gereken metotlar olmadığı için `private` yapıldı (02:37:45).

Ekrandaki son hal (ekrandan, 02:38:30):

```
private String addFooter( ) {
    return "Amount owed is " + getTotalCharge()+"\n"
        + "You earned " + getTotalBonus()
        + " frequent renter points";
}
private String statementLine( Rental aRental ) {
    return "\t" + aRental.getMovieTitle()+ "\t" +
        String.valueOf(aRental.getCharge()) + "\n";
}
```

Kod parçadır, derlenmedi. Testler geçti.

Hoca `Customer` sınıfının epey derlenip toparlandığını, en uzun metodun 10 satır kadar kaldığını, her alt iş kaleminin kendi `public` ya da `private` metoduna taşındığını söyledi (02:38:35). `Rental` çok sade bir ara sınıftır ve aracılığı ek kalabalık olmadan iyi yapar. `Movie`'de ise hâlâ Primitive Obsession ve ondan kaynaklanan `switch-case` vardır (02:39:03).

`fowler_09b` paketindeki `Movie` sınıfının başı (ekrandan, 02:40:00):

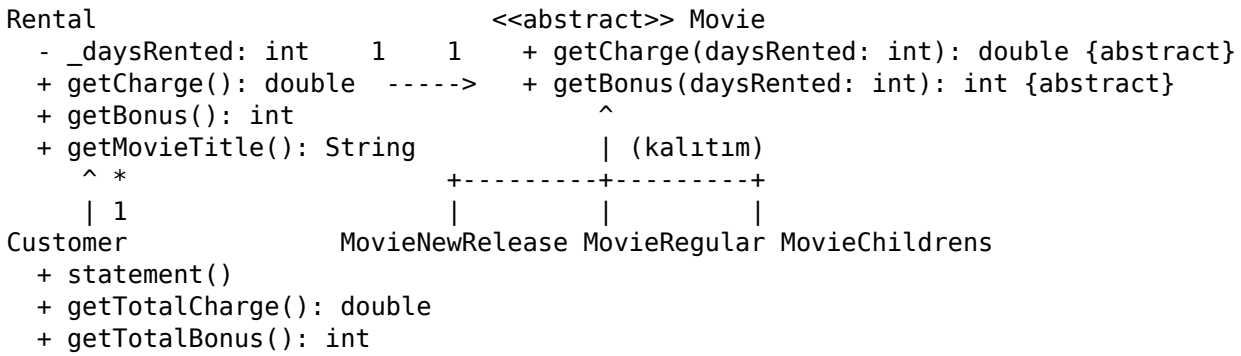
```
package fowler_09b;

public class Movie {
    public static final int CHILDRENS = 2;
    public static final int REGULAR = 0;
    public static final int NEW_RELEASE = 1;
    private String _title;
    private int _priceCode;
    public Movie(String title, int priceCode) {
        _title = title;
        _priceCode = priceCode;
    }
    public int getPriceCode() {
        return _priceCode;
    }
    public void setPriceCode(int arg) {
        _priceCode = arg;
    }
    ...
```

Kod parçadır, derlenmedi.

22. Onuncu Müdahale: Çok Biçimlilik ve Geriye Doğru Uyumluluk

Slayttaki tanım (ekrandan, 02:40:20): onuncu değişiklik, `getCharge` metodundaki `switch/case` ve `getBonus` metodundaki `if` deyimleri yerine çokbiçimlilik ilişkisi kullanılması (kalıtım ve soyut metotlar kullanılarak).



Primitive Obsession'ın çözümü burada tip kodunun yerine alt sınıflar koymaktır (02:39:36). Tek başına "düz" bir film türü olmadığı için `Movie` soyut sınıf yapılır ve `Childrens`, `Regular`,

`NewRelease` somut alt sınıflar olarak oluşturulur. `getCharge` ve `getBonus` metotları `Movie`'de soyut olur.

> **Ders sonrası not:** Hoca bu kısımda Primitive Obsession'ın çözümünü önce "Replace Type Code with Class" diye andı (02:39:36), ama anlattığı ve slaytta gösterilen tasarım (soyut `Movie` ve somut alt sınıflar) Replace Type Code with Subclasses'tır.

Soru (hocanın kendi sorusu): `Movie` neden arayüz (interface) değil de soyut sınıf yapıldı? **Cevap:** `Movie`'de hâlâ `title` üyesi ve onunla ilgili yapıcı (constructor) olacaktır. Sınıf belli bir iç mantık, yani üye alan ve üye metot içerdiği için arayüz yapılamaz, soyut sınıf olmak zorundadır (02:40:25).

Bu değişikliğin bedeli vardır (02:40:58). `Movie`'nin yapıcısına artık fiyat kodu gelmesine gerek kalmaz; çocuk filmi isteniyorsa doğrudan `MovieChildrens` nesnesi oluşturulur. Bu sınıfların kullanıldığı her yerde değişiklik gerekir: GUI formu, web servisi ya da veritabanıyla çalışan kod ve en azından test case'ler. Testlerin `setUp` metodunda filmler fiyat koduyla oluşturuluyordu; artık `matrix = new Movie(...)` yerine `matrix = new MovieRegular(...)` gibi yazılacaktır (02:42:02).

> **Hocanın vurgusu:** Refactoring'in katı tanımı, dışarıya açılan arayüzü değiştirmeden iç çalışma biçimini değiştirmektir. Bu eylemde dışa açılan arayüz değiştiği için önceki refactoring'lere göre geriye doğru uyumluluk kaybedilir; bu sizin için sorunsa bu son eylemi yapmayın (02:42:24). Ama bilin ki bu bir kod kusurudur (Primitive Obsession): ileride yeni film türleri desteklenecekse yazılımın birçok yerinde ek düzenleme gerekir. Eylem yapılırsa yeni bir alt sınıf eklemek her şeyi çözer, başka hiçbir yere dokunulmaz; test case'lere de yalnız yenisi eklenir (02:43:10).

Hoca bazı yazılımların bir noktada geriye doğru uyumluluğu bırakmasının bir nedeninin de bu olduğunu söyledi. Geriye doğru uyumluluk önemli değilse bu eylem yapılır, aksi halde yapılmaz. Bu adım derste kodlanmadı; ders kodlarında yapılmış hali bulunduğu söylendi (02:43:47).

23. Karşıt Kalıplar (Antipatterns)

Dersin sonunda hoca antipattern (karşıt kalıp) kavramına kısaca değindi (02:46:25). Kod kusurları ve onları çözen refactoring eylemleri, bir metodun bir kısmı ya da iki sınıfın birkaç metodu gibi küçük ölçekli düzenlemelerdir. Daha büyük ölçekte de kusurlar tanımlanmıştır; bunlara antipattern denir (02:47:04). Antipattern tasarım kalıbının tam karşıtıdır: güzel bir çözüm, bir tasarım kalıbı gibi görünür, ama yaklaşımın önerilerini çiğneyen yanlış önerilerle ortaya çıkmış bir kalıptır.

Küçük yanlışları bilmenin yararı olduğu gibi büyük yanlışları bilmenin de yararı vardır (02:47:45). Ölçek açısından:

- Kod kusurları basit kodlama düzeyine daha yakındır.
- Yazılım geliştirme antipattern'leri tasarım kalıplarıyla aynı ölçektedir; tasarım kalıplarının karşıtlarıdır.
- Daha üst düzeylerde yazılım mimarisi antipattern'leri ve yazılım proje yönetimi antipattern'leri (Software Project Management Antipatterns) tanımlanmıştır (02:48:13).

Hoca, refactoring ve kod kusurlarını anlatan kaynaklar gibi antipattern'ler için de bir kitap ve iyi çevrimiçi kaynaklar olduğunu söyledi. Derste kod kusurlarının ve refactoring eylemlerinin hepsi değil, önemli bir kısmı görüldü; daha fazlası için Fowler'ın kitabı ve çevrimiçi kaynaklar incelenebilir (02:49:01).

> **Ek bilgi (derste anlatılmadı):** Hocanın andığı antipattern kitabı, büyük olasılıkla, Brown, Malveau, McCormick ve Mowbray'in "AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis" (1998) kitabıdır. Bu kitap antipattern'leri yazılım geliştirme, yazılım mimarisi ve yazılım proje yönetimi olarak üç kategoride ele alır; hocanın saydığı üç düzey bununla örtüşür.

Örnek Sorular

Soru 1 (ekrandan, 01:50:00): Üçüncü adımın sonucunda `Rental` sınıfı yeni bir metot (`getCharge`) kazandı. Bu metot için yeni bir test case mi oluşturulmalı (beyaz kutu), yoksa

mevcut test case'lerle mi devam edilmeli (kara kutu)?

> **Derste verilen cevap:** Beyaz kutuyu seçmek test kalitesini artırır, ama sınırlı proje zamanının uzamasına yol açabilir. Kod kapsama hedefi projenin sınamaya ayırdığı zamana göre belirlenir. Örnek karmaşık olmadığı için mevcut test case'lerle (kara kutu) devam edildi.

Soru 2 (derste sözlü, 01:38:27): Bir metot içindeki değişkenlerin adlarını değiştirmek (`each` → `aRental`, `thisAmount` → `result`) bir refactoring eylemi sayılır mı?

> **Derste verilen cevap:** Sayılır; dış davranış değişmeden kodun anlaşılabilirliği artar. Eclipse'te Refactor > Rename ile yapılabilir.

Soru 3 (derste sözlü, 02:40:25): Onuncu müdahalede `Movie` neden arayüz değil de soyut sınıf yapılır?

> **Derste verilen cevap:** `Movie`'de `title` üye alanı ve onu kuran yapıcı kalacaktır. Üye alan ve somut metot içeren bir sınıf arayüz olamaz; bu yüzden soyut sınıf yapılır. `getCharge(int)` ve `getBonus(int)` ise soyut metotlardır.

Soru 4 (derste sözlü, 01:54:32 ve 01:56:10): Replace Temp with Query sonrasında ortalama test süresi 6 ms'den 10 ms'ye çıktı. Artış oranı nedir ve 4 kiralamada `getCharge` kaç kez çağrılır?

> **Çözüm (derste verilmedi):** Süre artışı $(10-6)/6 \approx \%66.7$; yeni süreye göre azalma $(10-6)/10 = \%40$. Hocanın söylediği %33 iki hesapla da tutmaz. `thisAmount` kaldırıldıktan sonra `getCharge` her kiralama için iki kez çağrıldığından 4 kiralamada $4 \times 2 = 8$ çağrı olur. Hesap python ile doğrulandı.

Dersin Toparlaması

Dersin ilk yarısında kod kusurları kataloğu tamamlandı. Large Class için Extract Class, Extract Subclass ve Duplicate Observed Data; değişikliğin dağınık yerlere dokunduğu Divergent Change ve Shotgun Surgery için Extract Class ve Move Method; Primitive Obsession için beş "Replace" eylemi (Data Value with Object, Type Code with Class, Type Code with Subclasses, Type Code with State/Strategy, Array with Object); son olarak Feature Envy ve onun çözümü olan Move Method ve Extract Method anlatıldı. Hoca her kusurda uyum ve bağlaşım ilkelerine dönerek eylemin neden iyi olduğunu gösterdi. Strategy ve Visitor gibi bilinçli tasarımların kusur sayılmaması gerektiğini vurguladı.

İkinci yarıda bu eylemler Fowler'ın video kiralama örneğinde birlikte kullanıldı. Uzun `statement` metodu Extract Method ile bölündü, adlar düzeltildi, Feature Envy gösteren hesaplar Move Method ile önce `Rental`'a sonra `Movie`'ye taşındı, geçici değişkenler Replace Temp with Query ve Extract Method ile kaldırıldı, nesneler arası iletişim katmanlandırıldı. Her adım ayrı bir pakette yapıldı ve JUnit testleriyle doğrulandı; derste iki kez test hatası çıktı ve testler bunları hemen yakaladı. Son adım olan çok biçimli `Movie` hiyerarşisi, geriye doğru uyumluluğu bozduğu için tartışılıp öğrencilere bırakıldı. Ders, büyük ölçekli kusurlar olan antipattern'lere kısa bir girişle kapandı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.